Progetto di Programmazione Avanzata

TeamManager – Sistema di gestione società sportive

Enrico Silvi

Corso di Laurea in Ingegneria Informatica

Anno Accademico 2025–2026

Sommario

1. Caso d'uso
2. Struttura progetto
    - 2.1 Architettura del sistema
    - 2.2 Server
        - 2.2.1 Sicurezza e autenticazione
        - 2.2.2 Modello dati
        - 2.2.3 Logica applicativa e servizi
        - 2.2.4 API
    - 2.3 Client
        - 2.3.1 Schermate e funzionalità
        - 2.3.2 Aggiornamento dati dal client
3. Scelte progettuali
4. Test
5. Futuri sviluppi
6. Principali strumenti utilizzati
7. Nota sull'uso dell'intelligenza artificiale

1 Caso d'uso

In base alla mia esperienza di allenatore di basket in alcune occasioni si è presentato il problema dell'impossibilità per un ragazzo di partecipare a una partita a causa di un certificato di idoneità agonistica scaduto. Questo è spesso dovuto alla difficoltà da parte delle società nella gestione delle scadenze dei certificati non permettendo di avvisare per tempo il giocatore. Questa applicazione vuole quindi fornire supporto in tal senso consentendo alle società la registrazione e gestione dei tesserati e dei loro certificati di idoneità agonistica.

Pertanto, con quest'applicazione l'utente (rappresentante della società) può:

- registrarsi e autenticarsi;
- creare la propria società sportiva;
- inserire nella squadra di appartenenza i giocatori tesserati nell'anno sportivo corrente;
- visualizzare tutti i tesserati in una tabella;
- filtrare la tabella per squadra;
- inserire e modificare la data di scadenza del certificato di un giocatore;
- escludere dalla società un giocatore che non ne fa più parte;
- verificare l'approssimarsi delle scadenze dei certificati .

2 Struttura progetto

2.1 Architettura del sistema

Il sistema è strutturato in una configurazione client-server, dove la comunicazione del client verso il server avviene esclusivamente attraverso richieste GET, POST e DELETE alle API esposte dal server stesso che a sua volta gestisce le comunicazioni da e verso il database MySQL.

2.2 Server

2.2.1 Sicurezza e autenticazione

L'implementazione della sicurezza degli accessi è stata realizzata tramite metodi di autenticazione personalizzata utilizzando componenti crittografici di Spring Security con l'aiuto di strumenti di intelligenza artificiale nello sviluppo. In fase di registrazione il sistema richiede all'utente oltre ad email e password, necessari successivamente per il login, anche nome, cognome e codice fiscale. Le password non vengono mai memorizzate in chiaro all'interno del database, ma sono protette da crittografia hash. Nei successivi login da parte dell'utente, il server restituisce al client un token di autenticazione, poi inviato nelle seguenti richieste tramite un header HTTP personalizzato (uso X-Token). Questo approccio consente al server di identificare l'utente autenticato evitando di richiedere ogni volta le credenziali.

2.2.2 Modello dati

Il modello dati è stato progettato nell'ottica di un possibile sviluppo futuro dell'applicazione. Sono presenti le seguenti entità:

- **Categoria**: permette di associare a una società le categorie a cui appartengono i propri giocatori;
- **CertificatoMedico**: associa ai giocatori i rispettivi certificati di idoneità agonistica;
- **Giocatore**: contiene i dati anagrafici dei giocatori;
- **IscrizioneStagionale**: consente di attribuire la categoria di appartenenza ai giocatori di una società;
- **Società**: contiene gli estremi anagrafici e fiscali della società;
- **Utente**: contiene i dati dell'utente memorizzati in fase di registrazione;
- **UtentiCategorie**: permette di profilare gli utenti (esclusi gli admin) ai fini dell'accesso ai dati del database;
- **UtentiSocietà**: associa gli utenti alle loro rispettive società.

L'entità "UtentiCategorie" non è al momento utilizzata, ma è stata inserita nella struttura del database nell'ottica di aggiornamenti futuri.

2.2.3 Logica applicativa e servizi

L'organizzazione della logica applicativa è strutturata nei seguenti Services elencati per ordine di utilizzo:

- **InitService**: crea le tabelle utilizzando il codice sql presente in `init.sql` nella cartella resources e, successivamente, le inizializza con dei dati di test presenti nel file `data.json` contenuto nella medesima cartella. Queste operazioni vengono eseguite utilizzando la libreria DataSource per la connessione al database e la logica descritta nei successivi Services;
- **AuthService**: esegue le operazioni di login, registrazione e logout degli utenti all'applicazione;
- **SocietaService**: permette all'utente di registrare la società all'interno del database associandola a sé stesso e di ottenere informazioni sulla società associata all'utente;
- **GiocatoreService**: consente all'utente di inserire o rimuovere i dati dei giocatori, aggiornare la data di scadenza del certificato di idoneità agonistica e di ottenere una lista dei giocatori della società contenente per ciascuno id, nome, cognome, codice fiscale, categoria e data di scadenza del certificato;
- **TokenService**: contiene la logica di creazione del token e una Map per associarlo all'utente attualmente autenticato.

2.2.4 API

La comunicazione con il server è permessa attraverso delle API. Gli endpoint per la comunicazione sono descritti all'interno di alcuni controller che richiamano le relative funzioni nei Services. Tutte le informazioni sono scambiate in formato JSON.

2.3 Client

L'applicazione lato client è organizzata in file FXML che descrivono la struttura delle schermate, in controller che descrivono le funzionalità in esse presenti e una classe ApiClient che si occupa della comunicazione con il server.

2.3.1 Schermate e funzionalità

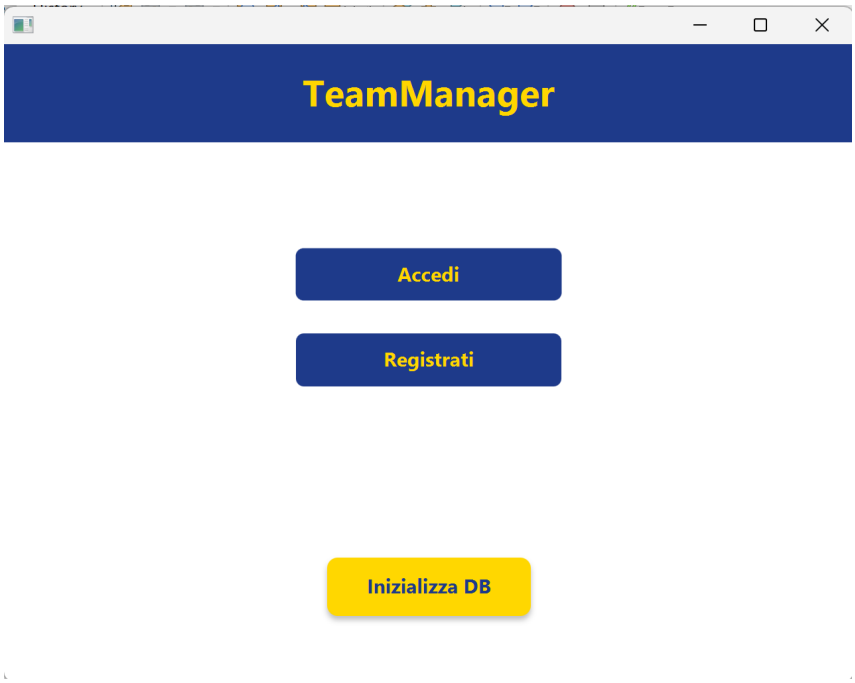

Figura 1: Schermata iniziale

La figura 1 rappresenta la schermata che appare all'apertura dell'applicazione. Da essa si può inizializzare il database attraverso il pulsante in basso oppure passare alle schermate di login e registrazione.

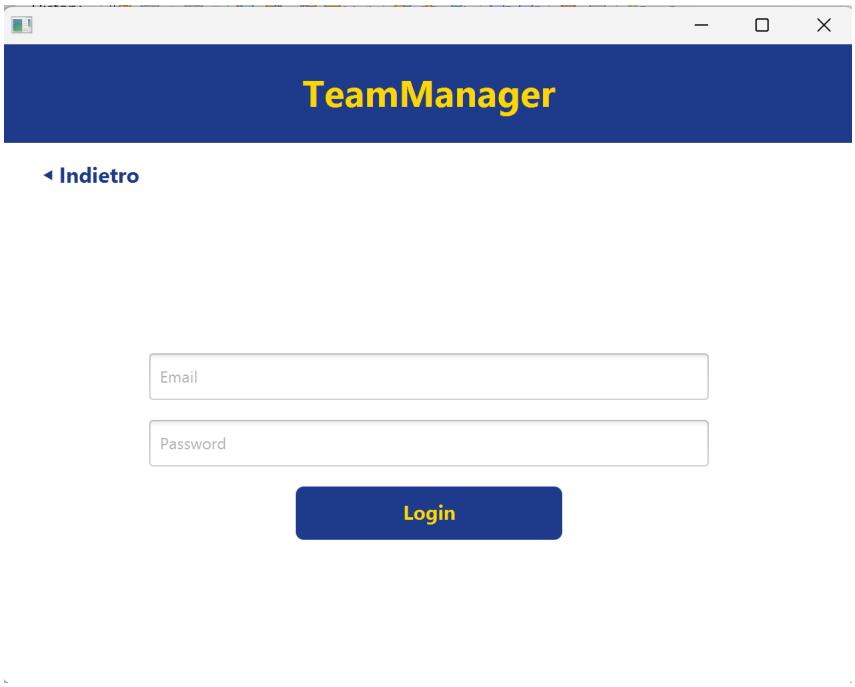

Figura 2: Schermata di login

In figura 2 è rappresentata la schermata di login del client che richiede email e password dell'utente per accedere al servizio. La schermata comprende, inoltre, in alto a sinistra, un pulsante per tornare alla schermata iniziale.

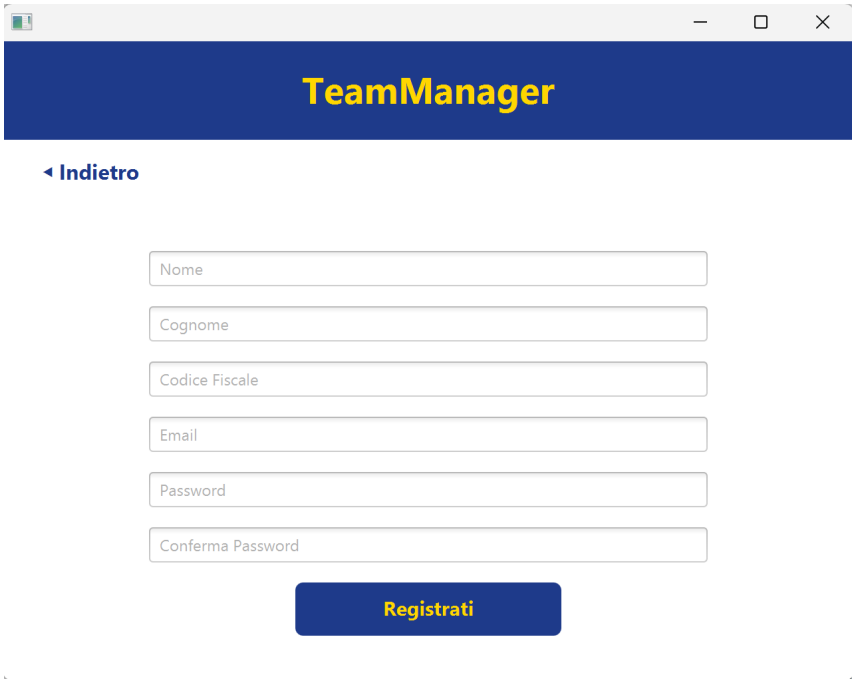

Figura 3: Schermata di registrazione

La schermata di registrazione (figura 3) richiede i dati necessari per la creazione del nuovo utente. Il client adotta prima dell'invio della richiesta al server un controllo sui dati inseriti. In particolare, controlla che il codice fiscale sia composto da 16 caratteri alfa-numerici e che l'email sia nel formato [userid]@[dominio].[estensione], dove userid, dominio ed estensione sono caratteri alfa-numerici.

Figura 4: Schermata di creazione della società

La schermata della figura 4 viene visualizzata dall'utente subito dopo la registrazione o nell'eventualità, dopo il login, l'utente non risulti ancora collegato ad una società. Il form della schermata richiederà il nome della società e la partita iva (non sottoposta a controlli) per la registrazione.

Nome	Cognome	Codice Fiscale	Squadra	Scadenza ce...	Giorni rima...
Keegan	Freeman	ABSJXI00G0050...	U15	2026-01-02	-39
Kenyon	Boyd	ABMQXI00G00...	Prima Squadra	2026-01-12	-29
Logan	Cobb	ABGWLI00G00A...	U14	2026-01-12	-29
Abbot	Peters	ABZDTU00G00F...	U13	2026-01-15	-26
Hop	Rios	ABKGKE00G001...	Prima Squadra	2026-01-24	-17
Hashim	Mckay	ABNBBO00G00...	Esordienti	2026-01-26	-15
Owen	Hinton	ABXBWE00G00...	U19	2026-01-27	-14
Phillip	Herring	ABXWVI00G00E...	Esordienti	2026-01-27	-14
Marvin	Sampson	ABHFNE00G003...	U17	2026-01-30	-11
Clarke	Berry	ABVRXA00G00...	U15	2026-01-31	-10
Stephen	Dyer	ABSDQO00G00...	U15	2026-02-01	-9
Hedley	Atkinson	ABXOFE00G007...	Prima Squadra	2026-02-13	3
Emmanuel	Gamble	ABCAQE00G00F...	U13	2026-02-13	3

Figura 5: Schermata home

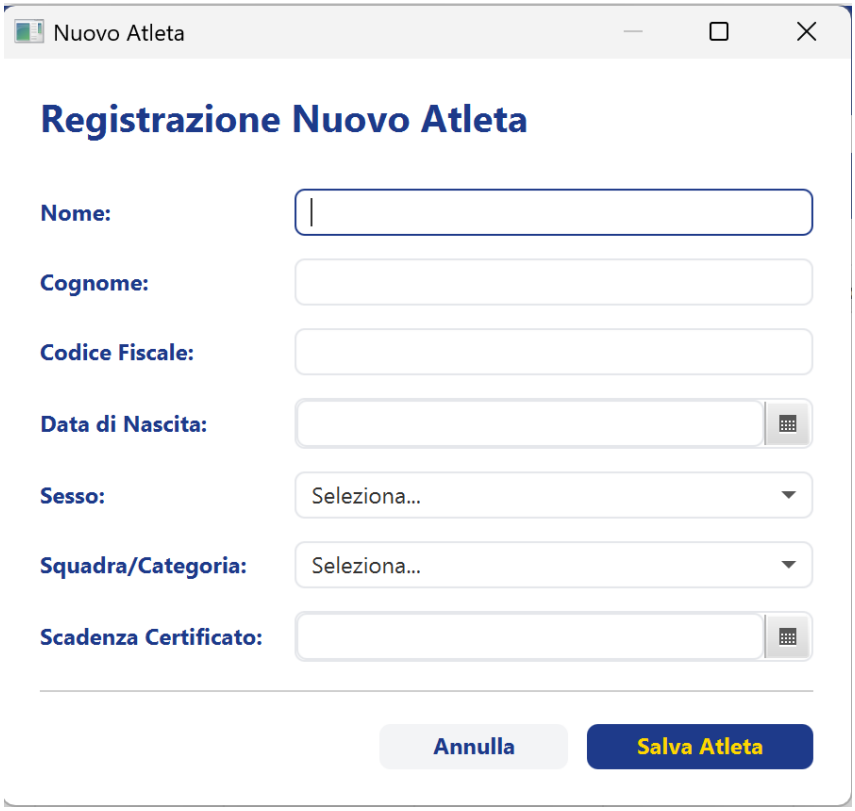

Figura 6: Schermata di aggiunta giocatore

Nella schermata principale (figura 5) dell'applicazione, possiamo trovare, nella zona alta, il nome della squadra e il tasto per effettuare il logout. Il resto della schermata è occupato da una tabella che contiene i dati dei giocatori della società, da un menù a tendina che consente di filtrare i giocatori per la squadra di appartenenza e un tasto che apre la schermata per aggiungere un nuovo giocatore. Infine, selezionando un giocatore, l'utente può eliminarlo o modificare la data di scadenza del suo certificato di idoneità agonistica.

La schermata in figura 6 consente la registrazione di un nuovo atleta e dei relativi dati.

2.3.2 Aggiornamento dati dal client

Il client comunica con il server solamente tramite le API esposte dal server al fine di:

- autenticare un utente;
- richiedere la lista dei giocatori della società;
- aggiungere o rimuovere giocatori;
- modificare date di scadenza dei certificati di idoneità.

3 Scelte progettuali

- utilizzo di Spring Boot lato server
- divisione della logica in più livelli (Controller, Service e Repository)

- uso dei DTO per la gestione dei dati passati tra client e server
- utilizzo di Spring Boot Security per una maggiore sicurezza nella gestione delle password
- creazione degli enum Categoria e Sesso per standardizzare gli ingressi

4 Test

Il server include anche dei test automatici, in particolare:

- test sulla connessione al database;
- test sulla funzione che calcola la stagione corrente.

È disponibile un account di prova con dati già generati con le seguenti credenziali:

- **mail**: test@test.it
- **password**: test

5 Futuri sviluppi

La struttura dell'applicazione e del server sono stati implementati in modo tale da facilitare eventuali sviluppi futuri. In particolare, è stata prevista la possibilità di aggiungere dei ruoli diversi all'interno della società, in modo che altri utenti possano accedere ai dati della società anche in parte limitata.

6 Principali strumenti utilizzati

Il progetto TeamManager utilizza diversi strumenti contenuti nel programma del corso, in particolare l'utilizzo di Spring Boot per la creazione del server e la creazione di applicazioni usando JavaFX.

Nota sull'uso dell'intelligenza artificiale

Per lo sviluppo di questo progetto è stato fatto uso dell'intelligenza artificiale, utile in particolare per la risoluzione di errori e per la migliore comprensione di alcuni concetti. L'AI è stata utilizzata per la creazione delle funzioni che gestiscono le richieste GET, POST e DELETE lato client. Inoltre, è stata utile nel fornire suggerimenti in relazione alla gestione dell'autenticazione tramite token e di come questi potessero essere trasferiti nelle richieste HTTP tra client e server. Anche la realizzazione dei Java Beans è stata ottenuta con i suggerimenti dell'AI.